RTos UVC 协议及代码介绍

V1.0.0

2023 年 8 月 3 日

上海富瀚微电子股份有限公司
地址：上海市徐汇区宜山路 717 号 2 号楼 5 楼 -6 楼邮编：200233
网址：http://www.fullhan.com
传真：021-64066786
电话：021-61121558
邮箱：sales@fullhan.com

版权所有 © 上海富瀚微电子股份有限公司。保留一切权利。
非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标申明

Fullhan®、**富瀚均为上海富瀚微电子股份有限公司的商标**

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意
由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

目录

目录 III

1 修订日志 1-1

2 前言 2-1

3 uvc 协议 3-1
- 3.1 uvc 概述 . . . 3-1
- 3.2 uvc 拓扑结构 . . . 3-1
  - 3.2.1 VC 组件 . . . 3-2
  - 3.2.2 操作模型 . . . 3-5
  - 3.2.3 VC 接口 . . . 3-6
  - 3.2.4 VS 接口 . . . 3-6
- 3.3 uvc 描述符 . . . 3-8
- 3.4 uvc 请求 . . . 3-9
  - 3.4.1 Set Request . . . 3-10
  - 3.4.2 Get Request . . . 3-11
  - 3.4.3 VC 控制请求 . . . 3-12
  - 3.4.4 VS 控制请求 . . . 3-14

4 Fullhan RTos 平台 uvc 介绍 4-1
- 4.1 uvc 初始化 . . . 4-1
- 4.2 视频格式描述符构建 . . . 4-2
- 4.3 Uvc 控制流程 . . . 4-4
  - 4.3.1 uvc 标准类请求控制流程 . . . 4-4
  - 4.3.2 uvc 扩展请求控制流程 . . . 4-5
- 4.4 格式切换控制流程 . . . 4-8
- 4.5 UVC 数据传输流程 . . . 4-11
  - 4.5.1 码流获取与编码 . . . 4-11
  - 4.5.2 YUV 码流处理 . . . 4-12
- 4.6 扩展功能 . . . 4-13

5 Uvc demo 的使用和配置方法 5-1

6 UVC api **6-1**
6.1 API 接口 6-1
6.1.1 fh_uvc_ops_register 6-1
6.1.2 fh_uvc_init 6-1
6.1.3 fh_uvc_stream_enqueue 6-2
6.1.4 fh_uvc_stream_pts 6-2
6.1.5 fh_uvc_status_get 6-2
6.2 结构体定义 6-4
6.2.1 struct uvc_format_info 6-4
6.2.2 struct uvc_frame_info 6-4
6.2.3 struct uvc_stream_ops 6-5
6.2.4 struct UvcInfo 6-6
6.2.5 struct uvc_request_data 6-7
6.2.6 UVC_FORMAT 6-7

1 修订日志

修订时间	修订版本	修订人	修订描述
2022-04-18	V1.0.0	mengqx352	简述 uvc 协议和 uvc 代码逻辑

版权所有 @ 上海富瀚微电子股份有限公司

2 前言

本文档简单的介绍 uvc(usb video class) 协议和 Fullhan uvc 的结构及流程，方便客户在应用、开发 uvc 产品时更快的了解 uvc 协议内容和 Fullhan uvc 的代码流程，减少学习和开发周期。

由于 uvc 是基于 usb 的一种类协议，因此学习 uvc 协议之前最好对 usb 的基本协议有所认知，以便理解如 host、device、描述符和请求等基本概念及 usb 的数据结构。

本文的对 uvc 协议只进行简要概述，如果个别概念不能理解或需要更深入学习及开发 uvc 产品，请参考 uvc 官方文档：https://www.usb.org/documents?search=video&type%5B%5D=55&items_per_page=50。

 版权所有 @ 上海富瀚微电子股份有限公司

3 uvc 协议

3.1 uvc 概述

UVC 是属于 usb 行业规范中的 usb 设备类规范，用于 usb 接口的视频设备的一个统一的数据交换规范。其中 uvc 官方协议文档 uvc-1.5 class specification 是对整个 uvc 协议的框架做了规范，usb_video_example 则规定了描述符等的数据结构和相关的一些其他规定。

3.2 uvc 拓扑结构

uvc 视频功能位于 usb 设备类结构中的接口层。它由多个接口组成，这些接口将相关管道分组，并使视频功能生效。

在 uvc 规范中明确要求，具有实际 uvc 功能的视频设备只需要两种接口：VC interface(视频控制接口) 和 VS interface(视频流接口)。Uvc 视频功能通过它的视频接口进行处理。每个 uvc 视频功能都有一个 VC 接口和多个 VS 接口，通过访问 VC 接口的设备控件实现对 UVC 设备的配置操作，而 VS 接口则用于负责传输视频数据流，两者相互配合完成 UVC 设备功能。

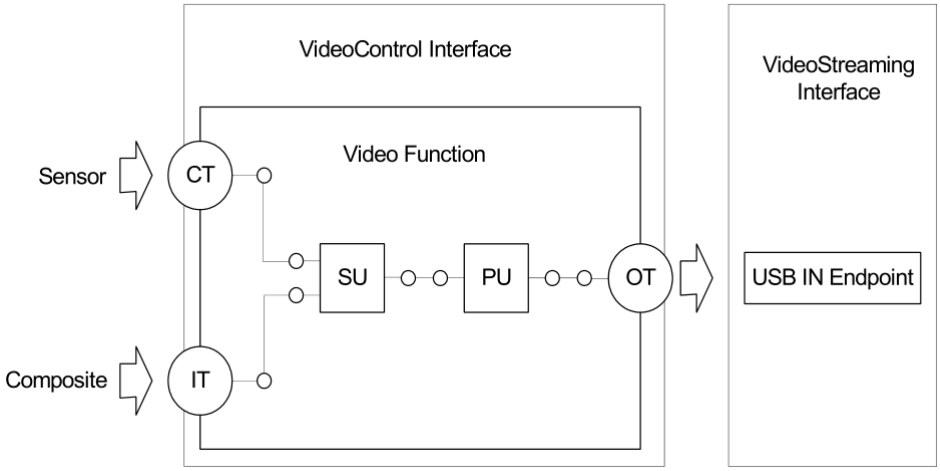
属于同一视频功能的单个 VC 和 VS 被称为视频接口集合 (VIC), 视频接口集合通过接口关联描述符 (interface Association Descrtptor) 进行描述。

UVC 将设备在拓扑结构上抽象成为几个主要的硬件功能模块，每个硬件功能模块规范了一些设备控件的配置支持，主要硬件模块有以下几种：

- 输入端 (input Terminal)
- 输出端 (output Terminal)
- Camera 端 (CameraTerminal)
- 选择单元 (selector unit)
- 处理单元 (processing unit)
- 扩展单元 (extension unit)

以 uvc 设备为例，其拓扑结构如图：

 版权所有 @ 上海富瀚微电子股份有限公司



图片 1: uvc 拓扑结构

3.2.1 VC 组件

单元或终端内部通过视频控件对其功能进行进一步的描述，控件通常提供对特定视频属性的访问。每个控件都有一组属性，可以对这些属性进行操作，或提供有关控件行为的附加信息。控件具有属性，这些属性可能包括：

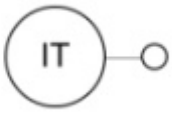
- Current setting
- Minimum setting
- Maximum setting
- Resolution
- Size
- Default

以 PU 单元的亮度控制为例，通过发出适当的请求，主机软件可以获取亮度控件的属性值，例如，可以在用户节目正确的显示亮度值。通过设置亮度控件的当前设置属性，主机软件可以更改正在播放的视频的亮度。

Input Terminal

作为数据流入 Video Function 的接收器，功能是作为已经被提取出来的数据的数据源，说白了就是 UVC 功能的数据源头。

输入端点通常在拓扑图中用下图表示：



图片 2: 输入端点

 版权所有 @ 上海富瀚微电子股份有限公司

Output Terminal

作为 Video Function 视频信息流出的出口，功能是作为输出数据的接收器。OT 只有一个输入 pin, 处理流程最后一个单元会将它的 out pin 与 OT 的 in pin 连接在一起。

输出端点一般在拓扑图中用下图表示：

图片 3: 输出端点

Canmer Terminal

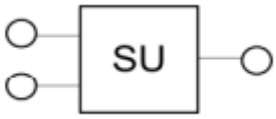
CT 代表硬件上的数据采集源 (sensor), 用于控制视频流传输中的设备特性的。它规定支持以下特性控制：

- Scanning Mode (Progressive or Interlaced)
- Auto-Exposure Mode
- Auto-Exposure Priority
- Exposure Time
- Focus
- Auto-Focus
- Simple Focus
- Iris
- Zoom
- Pan
- Roll
- Tilt
- Digital Windowing
- Region of Interest

Selector Unit

SU 单元用来控制图像数据采集源的切换和选择。例如上面相机拓扑图有两个数据输入源，这时需要 SU 来进行切换和选择。

选择器单元一般在拓扑图中用下图表示：



图片 4: 选择单元

 版权所有 @ 上海富瀚微电子股份有限公司

Processing Unit

PU 代表对采集所得图像数据进行加工的单元，负责对采集到的图像特性进行调整设置，主要有以下一些特性：

- Usr Controls
  - Brightness（背光）
  - Contrast（对比度）
  - Hue（色度）
  - Saturatio（饱和度）
  - Sharpness（锐度）
  - Gamma（伽马）
  - Digital Multiplier (Zoom)（数字放大）
  - Auto Controls
  - White Balance Temperature（白平衡色温）
  - White Balance Component（白平衡组件）
  - Blcklight Compensation（背光补偿）
- Other
  - Gain（增益）
  - Power Line Frequency（电源线平率）
  - Analog Video Standard（模拟视频标准）
  - Analog Video Lock Status（模拟视频死锁状态）

PU 单元在拓扑图中用下图表示：

图片 5: 处理单元

Encoding Unit

编码单元控制编码器的属性。它有一个输入和多个输出引脚。这些功能可以在流媒体开始之前或之后使用，支持以下功能：

- Select Layer
- Video Resolution
- Profile and Toolset
- Minimum Frame Interval
- Slice Mode

- Rate Control Mode
- Peak Bit Rate
- Quantization Parameter
- Synchronization and Long Term Reference Frame
- Long Term Reference Buffers
- Long Term Picture
- Valid Long Term Pictures
- LevelIDC
- SEI Message
- QP Range
- Priority ID
- Start or Stop Layer
- Error Resiliency

EU 单元仅适用于板载带有视频编码器的设备。EU 单元的符号如下图所示。

图片 6: 编码单元

Extension Unit

XU 是为了特定供应商添加规范而提供的单元。扩展单元可以有一个或多个输入引脚和一个输出引脚. 尽管通用主机驱动程序无法确定扩展单元中实现了什么功能，但它应向供应商提供的客户端软件报告这些扩展的存在，并提供一种方法，用于从客户端软件向该单元发送控制请求，并从该单元接收状态（Fullhan uvc 中升级和参数配置等自定义功能均基于扩展单元实现）。

扩展单元一般在拓扑图中用下图表示：

图片 7: 扩展单元

3.2.2 操作模型

一台设备可以支持多种配置，在每个配置中可以有多个接口，每个接口可能具有多个可选设置。这些接口可以共同属于同一复合设备中的不同功能。同一设备中可以存在

多个独立的视频功能。属于同一视频功能的接口被分组到由接口关联描述符描述的视频接口集合中。如果设备包含多个独立的视频功能，则必须有多个视频接口集合（以及多个接口关联描述符），每个集合都提供对其关联视频功能的完全访问。

作为复合设备的一个例子，以配备内置麦克风的台式摄像机为例，这样的设备可以配置为一个接口集合处理音频功能的配置和控制，另一个接口集合处理 uvc 视频方面的功能，uvc 接口集合包含两个接口，其中一个是 VC 接口，用于控制功能的内部工作，而另一个是 VS 接口，用于处理从摄像头视频子系统接收的数据流

如前所述，视频功能位于设备类层次结构中的接口级别。后面各节描述了单个视频控制接口和可选视频流接口，以及用于视频功能控制和数据流传输的相关端点。关联描述符 (IAD) 的含义和使用方法参考 uvc 协议第 2.4.1 节"Video Interface Collection"和第 3.6 节"Interface Association Descriptor"，本文不进行描述。

3.2.3 VC 接口

VC 主要用于控制 UVC 功能中的 Units 和 Terminals，通过对不同的 Units 和 Terminals 下发不同的指令，从而影响 UVC 设备的当前工作状态;Host 端下发的所有对 UVC 设备的操纵控制命令都是经由此 Interface 完成的，这个以上面拓扑图所表示的设备为例，该设备包含了一个 Processing Unit、一个 Input Terminal、Output Terminal 和 Selector Unit，VC Interface 需要对它们下发不同的命令，为此 VC Interface 需要包含一个控制端点 (强制性要求)，用的就是每个 USB 设备中默认的端点 0。另外一个 Interrupt 端点则是可选的，用来返回或通知 Host 端当前的 UVC 设备内部状态有变化，大部分情况下，一个 UVC 设备的 VC Interface 不需一定要实现此端点，但一旦 UVC 设备需要实现某些特定 feature 时，规范会强制性要求实现该 interrupt 端点，有关更多信息，请参阅 UVC 协议第 2.4.2.2 节"Status Interrupt Endpoint"。

有关控制接口的更多介绍包括状态中断端点、硬件触发中断、静态图片捕捉、光学变焦与数字变焦等，请参考 uvc 协议第 2.4.2 节"VideoControl Interface"。

3.2.4 VS 接口

VS 接口用于在主机和视频功能之间交换数字数据流，VS 接口是可选的。一个 UVC 视频功能可以有零个或多个与之相关联的 VS 接口，每个接口可以支持不同性质和格式的数据。每个 VS 接口可以有一个用于视频的 ISOC 或 BUIL 数据端点，以及一个用于与视频相关的静态图像的可选的专用 BULK 端点（请参阅 uvc 协议第 2.4.2.4 节"Still Image Capture"）。这种结构保证了 VS 接口与端点相关的单个数据流之间的一对一关系。

如果 VS 接口使用了 ISOC 端点，则具有 isoc 端点的接口必须要有至少一个可以用于改变接口和端点属性的可选配置，即同一个接口号具有不同的 alt 配置。可选的 alt 配置的典型用途是提供一种来更改活动 isoc 端点对 USB 带宽要求的方法，所有具有 isoc 端点的 uvc 视频设备必须包含一个零带宽的可选配置，且默认一定是 alt0。Alt0 接口不包含视频流 isoc 端点描述符。

 版权所有 @ 上海富瀚微电子股份有限公司

包含 bulk 端点的 vs 接口只支持 alt0，uvc 协议规定了 bulk 端点不允许使用其他的可选配置。当 bulk 端点仅用于静态图像传输（请参阅 uvc 协议第 2.4.2.4 节"Still Image Capture"），此限制不禁止批量端点和等时端点共存使用。在这种情况下，每个 alt 配置都将包括 isoc 端点和 bulk 端点描述符。

VS 接口相关的功能描述还包括 isoc 端点带宽的选择 (uvc 协议第 2.4.3.1 节"Stream Bandwidth Selection")、isoc 和 bulk 模式传输过程示例 (uvc 协议第 2.4.3.2 节"Video and Still Image Samples")、uvc 数据包头格式 (uvc 协议第 2.4.3.3 节"Video and Still Image Payload Headers") 和数据量同步及速率匹配 (uvc 协议第 2.4.3.4 节"Stream Synchronization and Rate Matching")。以上内容如须深入了解请参考 uvc 协议具体章节，这里仅简单的介绍一下 uvc 视频流数据的包头格式。

每个包含视频或静态图像样本数据的有效负载传输必须从有效负载报头开始。Uvc 数据报头格式:

offset	field	size	value	description
0	bHeaderLength	1	number	有效负载报头的长度（字节），包括此字段。
1	bmHeaderInfo	1	bitmap	提供有关标头后面的示例数据的信息，以及此标头中可选标头字段的可用性。D0：帧 ID--对于基于帧的格式，每当新视频帧开始时，该位在 0 和 1 之间切换。对于基于流的格式，该位在每个新编解码器特定段的开头在 0 和 1 之间切换。这种行为对于帧格式是必需的，对于流格式是可选的。对于基于流的格式，必须通过 Video Probe and Commit controls 的 bmFramingInfo 字段指示对该位的支持（参见第 4.3.1.1 节 ``Video Probe and Commit Controls"）。D1：帧结束 --如果当前的有效负载是当前视频帧或当前镜头图像的最后一个数据包时，此位置位，对于流视频格式，则为指定的特殊编码段的结束。对于所有有效负载格式，此行为都是可选的。对于流视频格式，必须通过 Probe and Commit controls 的 bmFramingInfo 字段指示对该位的支持（参见第 4.3.1.1 节 ``Video Probe and Commit Controls"）。D2：PTS--如果 uvc 的报文头部中需要包含 dwPresentationTime 字段，则设置此位。D3：参考时钟源 --如果 uvc 的报文头部中需要包含 dwSourceClock 字段，则设置该位。D4：有效载荷特殊位。有关使用方法，请参见自定义有效负载规范。D5：静态图像 --如果后面跟的数据是静态图像帧的一部分，则设置此位，并且仅用于静态图像捕获的方法 2 和 3(参阅 uvc 协议第 2.4.2.4 节)。对于临时编码格式，该位表示后续跟随数据是帧内编码帧的一部分。D6：错误 --如果此有效载荷的视频或静态图像传输中存在错误，则设置此位。D7：报头结尾 --如果这是数据包中的最后一个报头组，则设置该位，其中报头组指的是该字段以及由该字段中的位标识的任何可选字段（为将来的扩展而定义的）。

下面的字段是否会包含在 uvc 的报头格式中，由上述的报头 bmHeaderInfo 字段决定。

 版权所有 @ 上海富瀚微电子股份有限公司

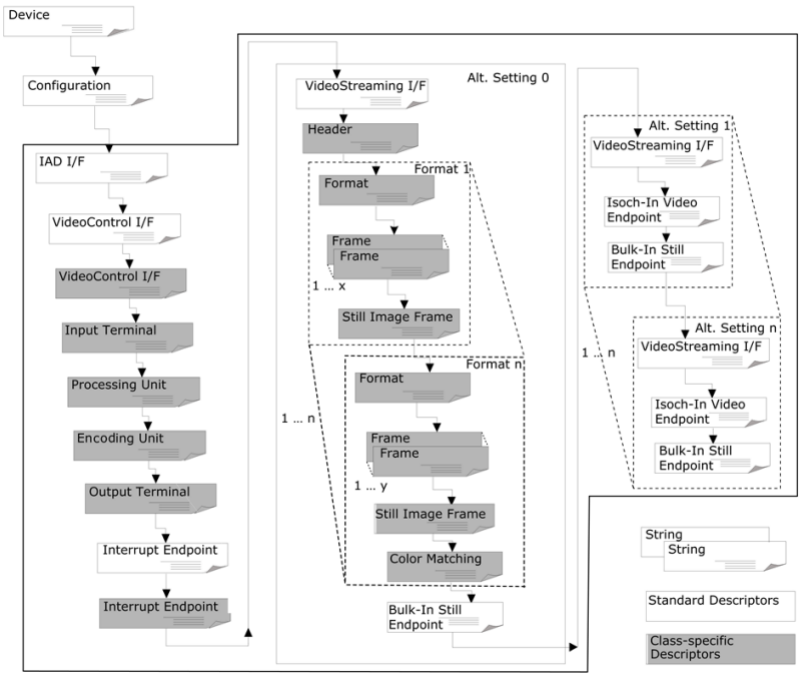
offset	field	size	value	description
Variable	dwPresentationTime	4	Number	演示时间戳（PTS）：设备端捕获视频原始帧开始时的源时钟时间（以设备时钟为单位）。对于一个视频帧分多个有效载荷传输的包头，可以重复该字段，该值在该视频帧中应保持不变。PTS 的单位与 Video Probe Control 响应的 dwClockFrequency 字段中规定的相同。
Variable	scrSourceClock	6	Number	由两部分组成的参考时钟源值。D31..D0：本机设备时钟单位中的源时钟 D42..D32: 1KHz SOF 令牌计数器 D47..D43：保留，设置为零。低 32 位（D31..D0）为从源系统时钟（STC）采样的时钟值。STC 的采样时间必须与 USB 总线时钟相关联。因此 SCR（D42..D32）最重要的 11 位包含 1 KHz SOF 计数器，代表 STC 采样时的帧号。

另外，有关 uvc 的控制传输和请求的处理流程，可以参考 uvc 协议第 2.4.4 节“Control Transfer and Request Processing”，本文不再描述。

3.3 uvc 描述符

USB 设备使用描述符来报告其属性，描述符是具有定义格式的数据结构。有关信息，请参阅 USB 规范 2.0 版第 9.5 节“Descriptors”描述。

以下部分介绍 uvc 接口类的标准描述符及 uvc 类描述符。下图列出了一个包含 isoc 端点和专用静态图片传输的 bulk 端点的 uvc 设备整体描述符概述。



图片 8: 描述符结构

Uvc 相关的描述符包括标准描述符和视频设备的特殊类描述符：

- 标准描述符

  - 设备描述符（Device Descriptor）
  - 设备限定描述符（Device QualifierDescriptor）
  - 设备配置描述符（Configure Descriptor）
  - 其余速度描述符（Other Speed Descriptor）
  - 字符描述符（String Descriptor）

- 特殊类描述符
  - 接口联合描述符（Interface Association Descriptor）
  - 视频控制接口描述符（VideoControl Interface Descriptor）
  - 视频控制端点描述符（VideoControl Endpoint Descriptor）
  - 视频流接口描述符（VideoStreaming Interface Descriptor）
  - 视频流端点描述符（VideoStreaming Endpoint Descriptor
- uvc 视频特殊类描述符
  - 输入端描述符（input terminal descriptor）
  - 输出端描述符（output terminal descriptor）
  - 相机端描述符（camera terminal descriptor）
  - 选择单元描述符（selector unit descriptor）
  - 处理单元描述符（processing unit descriptor）
  - 编码单元描述符（encoding unit descriptor）
  - 扩展单元描述符（extension unit descriptor）
  - 输入头描述符（Input Header Descriptor）
  - 视频格式描述符（Payload Format Descriptors）
  - 视频帧描述符（Video Frame Descriptor）
  - 静态图像描述符（Still Image Frame Descriptor）
  - 颜色匹配描述符（Color Matching Descriptor）

关于各类描述符的结构定义和具体含义,请参考 uvc 协议第 3.7.2 节“Class-Specific VC Interface Descriptor”。上述描述符在 Fullhan uvc 实现中基本都有用到，VC 接口内部结构的实现用到了 CT、PU、XU 和 OT 几个单元，其中 CT 和 PU 可以用于视频参数的条件，具体可以调节哪些参数可以参考 uvc 协议第 3.7.2.3 节“Camera Terminal Descriptor”和第 3.7.2.5 节“Processing Unit Descriptor”描述符中所描述的内容。RTos uvc 中用到了 XU 单元，XU 单元为扩展单元，用于实现厂家的自定义协议扩展，Fullhan uvc 中实现了在线升级、sensor hex 更新，以及 manufacture、product 等 usb 产品信息的在线更新配置功能，但是需要特定软件的支持。

3.4 uvc 请求

uvc 类请求主要用于设置和获取与视频相关的控件信息。这些控件分为两大类：如亮度、曝光、选择器位置等功能的控制和如当前帧速率等视频数据端点上的数据传输控

制。

视频控制请求。通过操做嵌入在视频功能单元和终端中的各个控件的属性来控制视频功能。UVC 的 VC 接口描述符描包含单元和终端描述符的集合，每个描述符指示其单元实体中存在哪些控件。视频控制请求始终指向 UVC 的 VC 接口。请求中包含单元 id、控件 id 等信息足够视频功能可以正确路由到特定的请求，以返回或配置正确的功能。

视频流请求。视频流接口的控制通过视频流接口的执行进行，其请求被定位到视频流所在的接口。

请求可以是强制性的，也可以是可选的。如 SET_CUR 可以通过 GET_INFO 确定其是否支持。如果视频功能不支持某个请求，则必须在 Host 向该功能发出该请求时通过 stall 来表明这一点。

3.4.1 Set Request

此请求用于设置视频函数实体内控件的属性。下图为设置请求的数据结构。

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100001	SET_CUR	See following paragraphs.	Entity ID and Interface.	Length of parameter block.	Parameter block.
00100010			Endpoint.		

图片 9: Set 请求结构

bmRequestType 字段表示这是一个设置请求（D7=0，bitmap 含义参考 usb 协议 9.3 节 table 9-2）。该请求是一个特定于类的请求（D6..5=01），可以指向视频控制接口或视频功能的视频流接口（D4..0=00001），或视频流接口的视频数据端点（D4..0=00010）。

bRequest 字段包含一个常量，用于标识要修改的寻址控件的哪个属性。对于设置请求，控制的属性只有：当前设置属性（SET_CUR）

如果设置请求寻址的控件或实体不支持修改某个属性，则当主机发送 set 请求试图修改该属性时，控制管道必须都该 set 请求返回 stall。Set 请求只支持 CUR 属性，不支持默认值、最大值最小值之类的修改。有关请求常量的列表，请参阅 uvc 协议第 A.8 节“Video Class-Specific Request Codes”。

wValue 字段的作用由 wIndex 字段中的值来决定。根据具体设置的功能不同，wValue 字段的布局会发生变化。在大多数情况下，wValue 的高字节用于控制选择器（CS），当某个终端或单元如 CT 中包含多个控件可以处理时，wValue 的高字节用于选择处理哪个控件。如果该单元仅包含一个控件，则无需指定控件选择器，该可用于传递其他参数。

wIndex 字段的低字节用于指定要寻址的接口号或端点，高字节用于指定单元或终端 ID 或零。只能寻址视频功能中现有的实体，并且只能使用存在的接口或端点号。如果请求指定了未知或非实体 ID 或未知接口或端点号，则控制管道必须返回 stall。

 版权所有 @ 上海富瀚微电子股份有限公司

set 请求的实际参数在控制传输的数据阶段传递。wLength 字段指定了参数块的长度。参数的数据格式由 bRequest 和 wIndex 字段决定。具体各个实体的控制数据结构，请参考 uvc 协议。

3.4.2 Get Request

此请求用于获取视频功能实体内特定控件的属性。

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10100001	GET_CUR GET_MIN GET_MAX GET_RES GET_LEN GET_INFO GET_DEF	See following paragraphs.	Entity ID and Interface	Length of parameter block	Parameter block
10100010			Endpoint.		

图片 10: Get 请求数据结构

bmRequestType 字段指定这是一个 GET 请求（D7=1）。它是一个特定于类的请求（D6..5=01），指向视频功能（D4..0=00001）的视频控制接口或视频流接口，或视频流接口的视频数据端点（D4..0=00010）。

bRequest 字段包含一个常量，用于标识要返回的寻址控件或实体的哪个属性。控件的可能属性包括：* 当前设置属性（GET_CUR）* 最小设置属性（GET_MIN）* 最大设置属性（GET_MAX）* 默认设置属性（GET_DEF）* 分度属性（GET_REF）* 数据长度属性（GET_LEN）* 信息属性（GET_INFO）

GET_INFO 请求查询指定控件的功能和状态。发出此请求时，wLength 字段的值应始终设置为 1 字节。返回的结果是一个位掩码，报告控件的功能。这些位被定义为：

Bit field	Description	Bit State
D0	1=Supports GET value requests	Capability
D1	1=Supports SET value requests	Capability
D2	1=Disabled due to automatic mode (under device control)	State
D3	1= Autoupdate Control (see section 2.4.2.2 "Status Interrupt Endpoint")	Capability
D4	1= Asynchronous Control (see sections 2.4.2.2 "Status Interrupt Endpoint" and 2.4.4, “Control Transfer and Request Processing”)	Capability
D5	1= Disabled due to incompatibility with Commit state.	State
D7..D6	Reserved (Set to 0)	--

图片 11: GET_INFO 结构定义

版权所有 @ 上海富瀚微电子股份有限公司

GET_INFO 中的是 D2（由于自动模式而关闭）和 D5（由于与提交状态不兼容而关闭）反映控件状态，其他位是能力位。当状态位改变时，能力位不应改变。例如，当控件设置为自动模式（D2 set）时，GET_INFO 中的位 D1 不得更新。如果有控制指令可以将 D2 位的状态修改，则状态修改后设备需要具有发送控制变更中断的能力（即状态中断端点），且必须设置 D3（自动更新控制）。如果有控制指令可以将 D5 位的状态修改，那么设备应该具有发送控制变更中断的能力。

如果 EU 单元实现了可以控制修改最小或最大设置属性的功能，那么设备应该发送控制更改中断，以通知主机获取新的 GET_MIN 或 GET_MAX 值，因此必须设置 D3（自动更新控制）。

设备通过其 GET_DEF 值表明单元、终端和接口控件的硬件默认值，主机可以使用这些默认值将控件还原为默认设置。

如果寻址的控件或实体不支持读取某个属性，则在尝试读取该属性时，控制管道必须返回 stall。有关请求常量的列表，请参阅 uvc 协议第 A.8 节“Video Class-Specific Request Codes”。

同 set request 一样，wValue 字段的具体含义有具体的控件决定，请参考 uvc 协议第 4.2 节“VideoControl Requests”。

wIndex 字段和 wLength 字段同 set request 的含义和用法一致。

3.4.3 VC 控制请求

本节描述可用于操纵视频功能通过其 VC 接口和该接口所包含的单元支持的视频控件的可能请求。VC 接口的 Set 和 Get 请求都使用相同的参数块布局。Uvc 协议第 4.2 节"VideoControl Requests"描述了每个每个控件的请求格式及是否是强制性或可选的，对于特定的控件请求，如果向需要指定更详细的请求信息和数据格式，可参考 uvc 协议第 4.2 节“VideoControl Requests”。任何实现的 VC 接口或控件请求都必须符合该控件的定义。

接口控制请求

接口控制请求属于 VC 接口结构中的接口曾，这些请求用于设置或读取 VC 接口控件的属性:

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100001	SET_CUR	CS	Zero and Interface.	Length of parameter block.	Parameter block.
10100001	GET_CUR GET_MIN GET_MAX GET_RES GET_INFO				

图片 12: 接口控制请求

接口控制请求的控件目前只有电源模式控制 (详见 uvc 协议第 4.2.1.1 节"Power Mode Control") 和错误码控制两种（详见 uvc 协议第 4.2.1.2 节"Request Error Code Control"）。

错误码控制请求是一个只读的请求，当主机向 uvc 设备发送的某个请求被 stall 后，主机会紧着这发送接口错误码请求以获得上一个请求被 stall 的原因。错误码请求数据结构如下：

Control Selector		VC_REQUEST_ERROR_CODE_CONTROL		
Mandatory Requests		GET_CUR, GET_INFO		
wLength		1		
Offset	Field	Size	Value	Description
0	**bRequestErrorCode**	1	Number	0x00: No error 0x01: Not ready 0x02: Wrong state 0x03: Power 0x04: Out of range 0x05: Invalid unit 0x06: Invalid control 0x07: Invalid Request 0x08: Invalid value within range 0x09-0xFE: Reserved for future use 0xFF: Unknown

图片 13: 控制接口错误码

端元或终端控制请求

单元或终端控制请求用于 VC 接口内的各个单元或终端内的控件的控制，用于设置或读取视频功能单元或终端内部控件的属性:

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100001	SET_CUR	CS	Unit or Terminal ID and Interface	Length of parameter block	Parameter block
10100001	GET_CUR GET_MIN GET_MAX GET_RES GET_INFO GET_DEF				
00100001	SET_CUR_ALL	Set to 0	Unit or Terminal ID		
10100001	GET_CUR_ALL GET_MIN_ALL GET_MAX_ALL GET_RES_ALL GET_DEF_ALL				

图片 14: VC 接口内实体控件请求

其请求数据结构如上图所示，具体结构及各个控件的用法请参考 uvc 协议第 4.2.2 节“Unit and Terminal Control Requests”。

关于单元或终端请求的其他说明，如果控件支持 GET_MIN、GET_MAX 和 GET_RES 请求，则 MAX、MIN 和 RES 的值应受到约束，以满足（MAX-MIN）/RES 为整数。此外，CUR 值（由 GET_CUR 返回，或通过 SET_CUR 设置）也应受到约束，以满足（CUR-MIN）/RES 为整数。如果 SET_CUR 操作中提供了无效的 CUR 值，则设备应按照协议 stall 该请求，并用 0x04``Out of Range" 更新请求错误代码控制（参见第 2.4.4 节“Control Transfer and Request Processing”）。

3.4.4 VS 控制请求

视频流请求可以定向到视频流接口或相关的视频数据端点，具体取决于要操作的控件的位置。其视频流功能接口控件属性的设置和读取的请求如下：

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100001	SET_CUR	CS	Zero and Interface	Length of parameter block	Parameter block
10100001	GET_CUR GET_MIN GET_MAX GET_RES GET_DEF GET_LEN GET_INFO				

图片 15: 视频流接口请求

 版权所有 @ 上海富瀚微电子股份有限公司

视频流接口控件允许主机查询和设置与视频流格式和视频流编码相关的参数。这些参数包括视频流的格式、幅面大小和帧速率，以及与视频流相关联的可捕获的静止图像的格式和幅面。对于支持主机可调视频流编码参数的设备，还支持调整关键帧速率和压缩质量等参数。视频流接口只支持流错误代码的中断控制。

视频的探测和提交控制

视频码流参数的选择过程基于主机和视频流接口之间的共享协商模型，并考虑以下特征：

1. USB 的共享特性
2. 流媒体参数的相互依赖性
3. 有效载荷独立性
4. 传输期间码流参数的修改

协商模型由 Video Probe and Commit 控件支持。用 Probe 控件来检索和协商流参数，当获得可接受的流参数组合时，用 Commit 控件来配置 Probe 控件协商的参数配置硬件。

视频的探测和提交控件结构相对复杂且内容较多，本文不再描述，如须更深入的学习和了解 Video Probe and Commit 控件的数据结构和使用方法流程请参考 uvc 协议第 4.3.1.1 节“Video Probe and Commit Controls”。

另外，关于视频流接口的请求控制还有更多进阶内容，如静态图像的探测和提交控制（参阅 uvc 协议第 4.3.1.2 节“Video Still Probe Control and Still Commit Control”）、同步延时控制（参阅 uvc 协议第 4.3.1.3 节“Synch Delay Control”）、静态图像触发控制（参阅 uvc 协议第 4.3.1.4 节“Still Image Trigger Control”）、生产关键帧控制（参阅 uvc 协议第 4.3.1.5 节“Generate Key Frame Control”）、更新帧段控制（参阅 uvc 协议第 4.3.1.6 节“Update Frame Segment Control”）和码流的错误代码控制（参阅 uvc 协议第 4.3.1.7 节“Stream Error Code Control”），这些控制的应用属于 uvc 协议的更进阶的应用，本文只介绍简单的 uvc 协议原理，方便用户快速入门。

 版权所有 @ 上海富瀚微电子股份有限公司

4 Fullhan RTos 平台 uvc 介绍

前面的章节简单的介绍了 uvc 协议的结构、描述符和请求的有关知识，下面将结合代码解析 Fullhan uvc 代码的实现，及 uvc 数据和控制流程。RTos uvc_demo 代码中除 uvc 外还复合了 uac、cdc 和 hid 功能，在这里不进行更多的复合功能的解析，感兴趣可以去看代码自行学习，uac、cdc 和 hid 功能相比于 uvc 较为简单。本文只介绍 uvc 的功能逻辑和流程。

Rtos uvc 从码流格式方面来讲支持 NV12、YUY2、MJPEG、H264、H265 共 5 种视频格式，分辨率和帧率的支持可以通过幅面结构体（uvc_frame_info）配置自行增加或减少，通过格式结构体（uvc_format_info）和幅面结构体配置视频的格式、分辨率和帧率，代码会自定生产 VS 接口的视频格式和幅面描述符。另外支持 ISOC 端点和 Bulk 端点，二者可选择其中一种方式作为码流数据传输端点。ISOC 端点支持高带宽模式，相较于普通带宽模式，高带宽模式的传输速率增加了 3 倍，最高可以达到 1024*3*8000 字节（isoc 包每次最大 1024Byte，高带宽是 3 倍，USB2.0 的 isoc 包 125us 一个，每秒 8000 个），约等于 24MB/S，支持多种可选的 alt 带宽配置，可通过幅面配置结构体选择使用的可选带宽的 alt 号。另外实现了最多两个 VS 接口的配置，可以实现同时两种不同格式视频的 uvc 传输。

从控制方面来说，uvc 的 VC 接口由 CT、PU、XU 和 OT 四个虚拟实体组成，其中 CT 实现了曝光、聚焦、缩放等 CT 终端支持的各个控件的支持，可通过注册回调的方式实现对 sensor 各参数的控制，PU 单元方面的亮度、对比度、色度等视频显示参数的控件也通过注册回调的方式进行了支持。

RTos uvc 还实现了一些基于 xu 单元的自定义扩展功能，包括在线更新 sensor 的 hex 效果参数、在线升级、获取当前股价版本信息、固件导出、uvc 产品信息的获取和更新功能，但基于 xu 的自定义功能都需要基于 fullhan 开发的特定才可以正常使用。

下面将具体介绍每个 uvc 功能的实现流程。

4.1 uvc 初始化

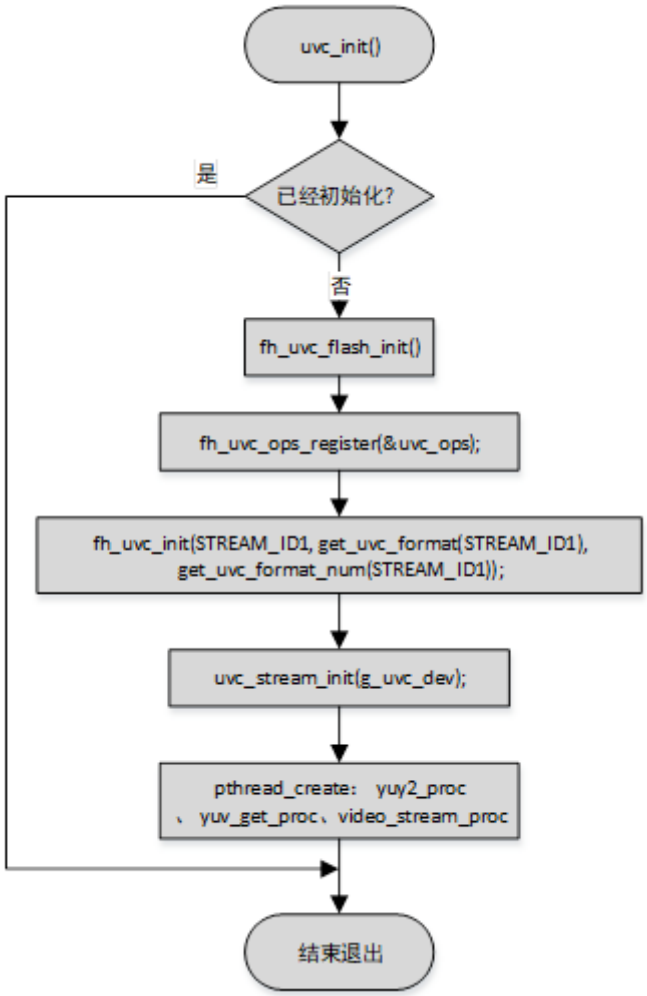
uvc 初始化过程：

1. fh_uvc_flash_init，初始化 flash 操作相关函数和线程，用于对 uvc 的扩展功能的支持，扩展功能基本上都需要对 flash 进行操作。
2. fh_uvc_ops_registor，注册 uvc 的回调函数，入参结构体包括格式切换回调函数、扩展功能处理函数、码流开关回调和各个控件参数处理回调等，具体参见代码。
3. fh_uvc_init，初始化 uvc 驱动，入参是 uvc 视频格式结构体和格式结构体数量，实现 uvc 驱动和驱动数据结构初始化，注册 uvc 支持的视频格式，初始化 usb 控制器并使之与 host 进行枚举。

 版权所有 @ 上海富瀚微电子股份有限公司

4. uvc_stream_init，初始化芯片的 dsp、isp、sensor 等与出图相关的硬件和驱动。
5. 创建 yuy2_proc、yuv_get_proc 和 video_stream_proc 三个线程，其中 yuy2_proc 线程用于将 nv12 数据转换为 yuy2 数据，yuv_get_proc 线程用于获取 yuv 码流数据，video_stream_proc 线程用于获取 host 请求格式的码流数据并将数据传输到驱动层。之所以存在 yuy2_proc 和 yuv_get_proc 是为了增加 YUY2 格式的传输效率，以空间换时间，防止 CPU 因获取码流导致的线程阻塞或 YUY2 数据转换导致数据传输延时过大的问题。

主要流程如下：



图片 16: uvc 初始化流程

4.2 视频格式描述符构建

RTos uvc 通过 demo 中头文件 format.h 来配置 uvc 支持的适配格式、幅面、帧率和 alt 带宽，如图下图所示。其中 uvc_format_info 结构体中定义了各种支持的视频格式和该格式中默认的幅面 id，以 mjpeg 格式为例，图中第一行第一个参数

V4L2_PIX_FMT_MJPEG 为 mjpeg 格式标识符，第二个参数 uvc_fream_mjpeg 为 mjpeg 格式的幅面信息结构体，第三个参数 6 为默认的 mjpeg 幅面是 uvc_fream_mjpeg 结构体中第 6 个幅面。uvc_fream_info 结构体定义了某种格式视频所能支持的幅面大小、该幅面支持帧率和该幅面申请的带宽，最后一个参数申请该幅面带宽的数字是 uvc 码流接口的可选配置的 alt 号，填写该 id 前应首先确定 uvc 视频接口各个 alt 所支持的带宽和该幅面码流所需要的带宽，该 uvc 的实现中没有实现根据码流带宽自动选择 alt 带宽的功能，需要通过该参数手动指定带宽。

```
static const struct uvc_frame_info uvc_frames_h265[] = {
    {640, 360,   {FPS(30), FPS(25), FPS(15), 0 }, 7},
    {640, 480,   {FPS(30), FPS(25), FPS(15), 0 }, 6},
    {1024, 576,  {FPS(30), FPS(25), FPS(15), 0 }, 5},
    {1280, 720,  {FPS(30), FPS(25), FPS(15), 0 }, 4},
    {2560, 1440, {FPS(30), FPS(25), FPS(15), 0 }, 3},
    {1920, 1080, {FPS(30), FPS(25), FPS(15), 0 }, 4},

    { 0, 0, { 0, }, 0},
};

static const struct uvc_format_info formats[] = {
    {V4L2_PIX_FMT_MJPEG, uvc_frames_mjpeg, 6},
    {V4L2_PIX_FMT_YUY2, uvc_frames_yuy2, 5},
    {V4L2_PIX_FMT_NV12, uvc_frames_nv12, 5},
    {V4L2_PIX_FMT_H264, uvc_frames_h264, 5},
#if !defined(CONFIG_ARCH_FH8626V100)
    {V4L2_PIX_FMT_H265, uvc_frames_h265, 5},
#endif
};
```

图片 17: format 结构体配置

fh_uvc_init(stream_id, uvc_format_info, format_num) 函数将 uvc 的格式结构体和格式结构体的数量传入 uvc 驱动层；

uvc 驱动中按照用户层传入的 format 结构体信息，分别挨个创建格式描述符和该格式下支持的各种幅面以及该幅面支持的各种帧率配置，完成所有格式描述符构建后按照 UVC 各种描述符顺序进行组合，重新计算 UVC 描述符总长度 VS 接口描述符总长度，达到 UVC 描述符更新的目的，当设备与主机建立连接后，进入枚举阶段，设备会将这些描述符按照 uvc 描述符的结构顺序发给主机，以向主机声明该设备所能够支持的各种能力。

 版权所有 @ 上海富瀚微电子股份有限公司

4.3 Uvc 控制流程

4.3.1 uvc 标准类请求控制流程

Uvc 控制流程是 uvc 中最重要的功能部分，本节介绍 VC 接口的控制流程。

一个控制指令由 host 主机通过控制端点 0 发出，uvc 设备的 usb 控制器收到控制指令后触发 usb 中断，并由中断处理函数对请求进行分类，标准 usb 请求逐层向控制器驱动层、usb core 层传递，如果是 uvc 类协议，则继续向 uvc 驱动层传递，最后通过调用具体的功能函数实现控制或请求数据的功能。下面以 uvc 类请求的亮度调节为例进行说明。

亮度值获取请求数据格式如下：

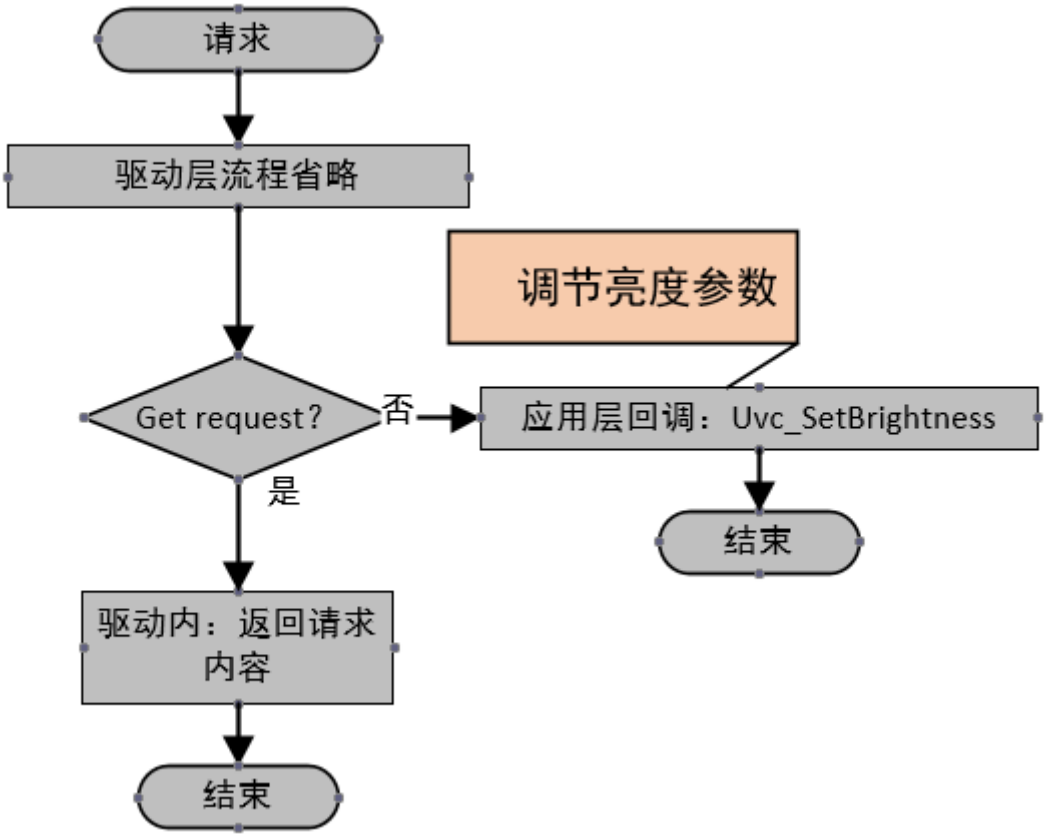
22	00	Get Def	ProcessingUnitID=2 CS=Brightness(2) (80 00)
22	00	SETUP txn	A1 87 00 02 00 02 02 00
22	00	IN txn [1 POLL]	80 00
22	00	OUT txn	

图片 18: 亮度请求命令

图中的请求是 GET_DEF，整个请求周期分为命令包、数据包和状态包，分别对应图中的 setup、in、out 三个阶段。其中 setup 即为请求指令，根据本文第 1.3.3.2 节 ``**端元或终端控制请求**" 中的请求数据格式进行解析，bmRequestType=0xA1 是属于 VS 接口的类请求；bRequest=0x87 是 GET_DEF；wValue=0x0200 表示 CS 为 2；wIndex=0x0200 低字节为 0 代表处理的接口接口号为 0，uvc 接口 0 为 VC 接口，接口 1 为 VS 接口，高字节为 0x02 表示 VC 接口内部实体号 id 为 2 的单元或中断，RTos uvc 代码中 PU 单元的 id 为 2；因此该指令的含义是获取 uvc VC 接口内 PU 单元的 CS 为 2 的控件的默认值，即获取亮度的默认值。

命令处理流程：

 版权所有 @ 上海富瀚微电子股份有限公司



图片 19: 标准类请求流程

上图描述了请求命令的传输和调用过程，对于标准的 uvc 类请求，仅设置属性会通过回调的方式调用到应用层代码，对图像属性进行调节，其他标准请求均在 uvc 驱动内完成。

当 uvc 控制器收到控制指令后，触发中断调用中断处理函数，然后向上逐层调用到 usb core 层对该请求进行解析，usb 标准请求在 core 层即可处理完毕并返回主机的请求数据，usb 的类请求则会根据接口号调用到每个接口的请求处理函数，如果是 Get 请求，则会返回请求值和请求的数据长度，之后发送到 usb 控制器，并由 usb 控制器传输给 host 主机，状态阶段的处理在控制器驱动中进行处理，uvc 开发者无需关心。

如果是 Set 的请求（目前仅 SET_CUR），则先解析请求内容，然后结束设置的具体数据，然后根据控件类型调用对应的 uvc 回调函数对 uvc 参数进行调节控制，并在驱动内部更新该控件 CUR 值，这些能够实现具体参数控制的回调函数通过 fh_uvc_ops_register 注册到 uvc 驱动层。

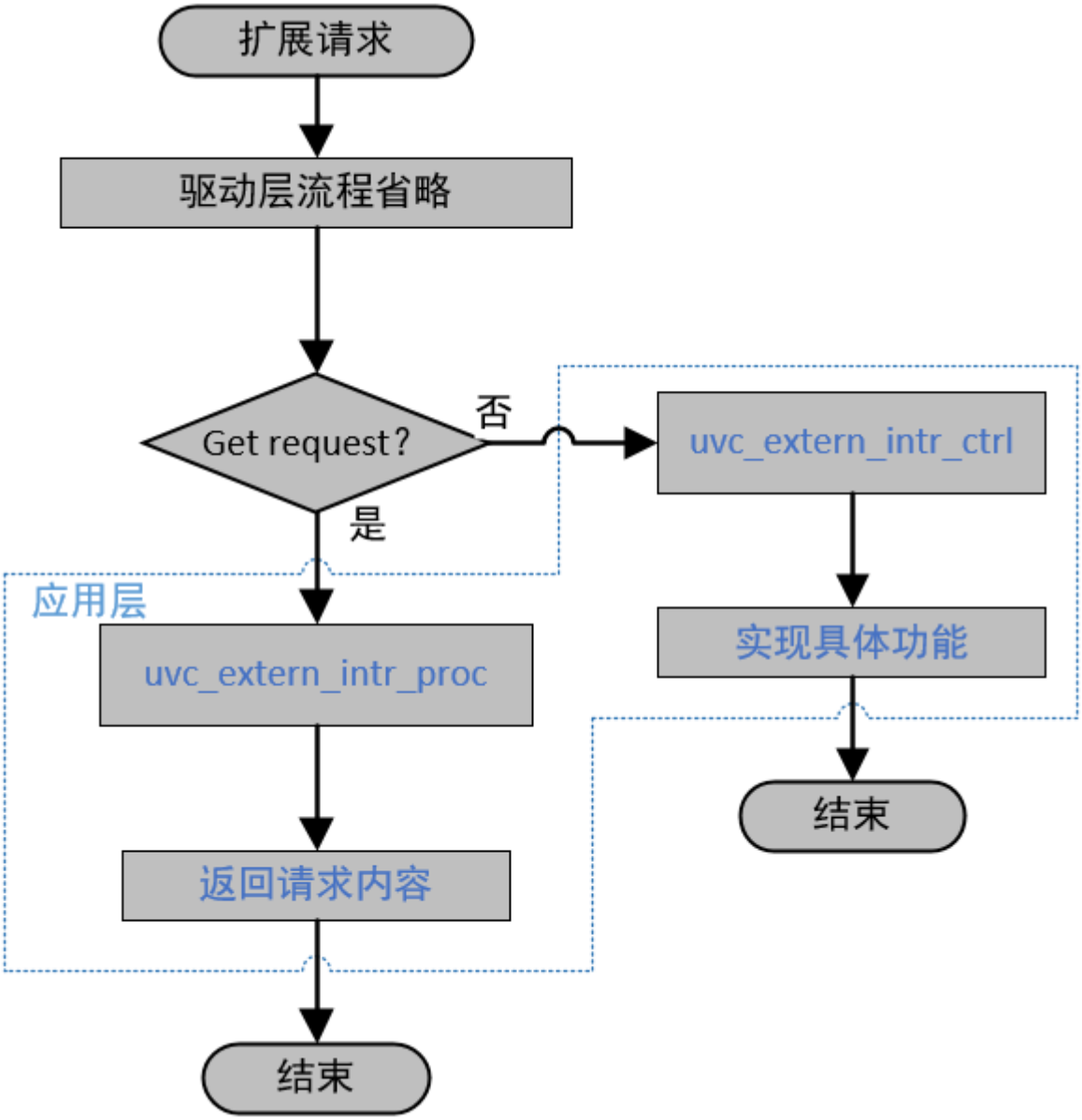
4.3.2 uvc 扩展请求控制流程

Uvc 扩展请求主要用于支持 uvc 扩展协议的一些自定义功能，如已经实现的 uvc 升级、sensor hex 升级、uvc 固件导出、uvc 版本读取、uvc 厂家信息更新等功能。其 Set 和 Get 请求的处理流程和 uvc 的标准类请求基本一致，不同的是当请求 wIndex 字段高

字节 id 为 XU 单元时，表示该请求为扩展请求，且扩展单元的 SET_CUR 和所有 Get 请求的具体处理函数通过回调调用到应用层，Get 请求将最终调用函数 uvc_extern_intr_proc 函数，SET_CUR 将调用 uvc_extern_intr_ctrl 函数，这两个函数通过 fh_uvc_ops_register 函数注册到 uvc 驱动层。将它们放在应用层实现的目的是方便用户扩展自定义功能在应用层进行开发时，尽可能的不去修改驱动。

注意：uvc_extern_intr_proc 和 uvc_extern_intr_ctrl 都是中断回调函数，不能有阻塞和耗时较多的处理。

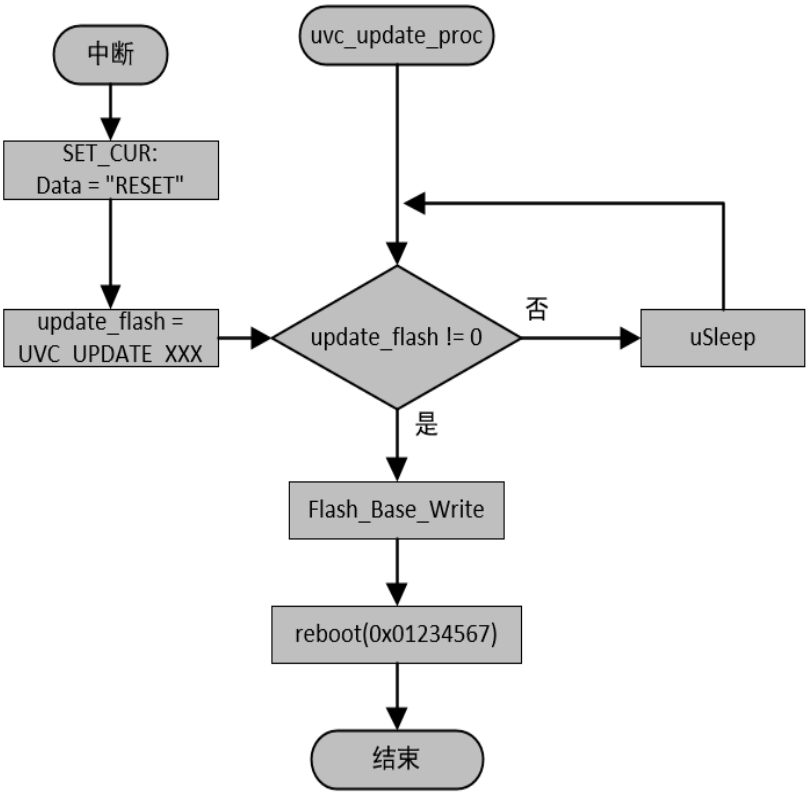
扩展请求的处理流程如下：



图片 20: 扩展控制请求流程

 版权所有 @ 上海富瀚微电子股份有限公司

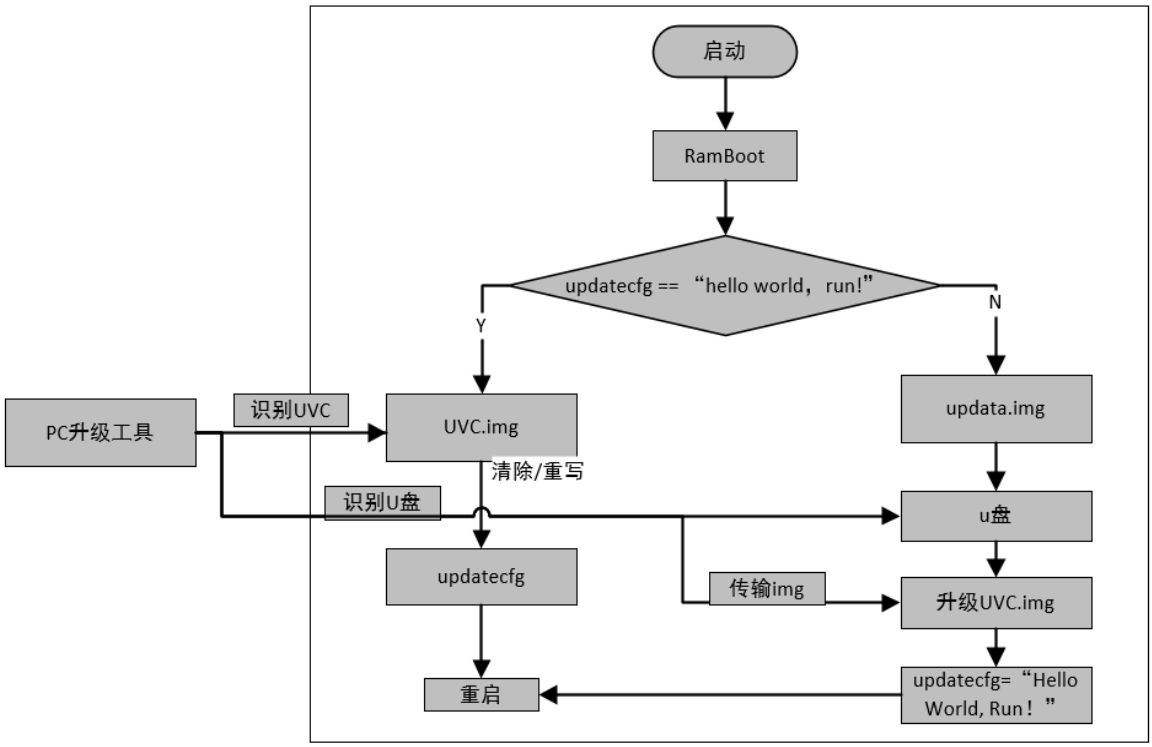
该流程图只展示了扩展请求的函数调用过程，具体扩展功能的功能流程并没有体现在流程图中。具体功能流程以升级功能为例，其流程如下：



图片 21: uvc 升级流程

当在主机上使用 uvc 升级软件升级时，主机向 uvc 设备发送 SET_CUR 请求，该请求不进入 uvc_extern_intr_pro 回调，随后发送升级请求的配置值，如果以 U 盘的方式进行升级则发送的数据为 ``RESET" 字符串，如果以串口方式升级则发送数据为 ``COMUPDATE" 字符串，如果以 hid 方式升级，则发送数据为 ``HU" 字符串，以 u 盘升级方式为例，收到 SET_CUR 的跟随数据后进入 uvc_extern_intr_ctrl 回调，并对结束的数据进行解析，U 盘升级方式则配置 update_flag=UVC_UPDATE_UDISK，随后升级线程检查到 update_flag 置位，则将升级字符串写入到 flash 的对应位置（即下图的 updatecfg，写入之后 updatecfg = ``UDISK UPGRADE"），之后重启 RTos 系统。重启后由 UVC 专用的 Ramboot 判断是否进入升级镜像，升级镜像为专用于 uvc 升级的一个 RTos 系统，与 UVC RTos 系统共存，并由 Ramboot 决定启动后进入哪一个系统，如果进入升级镜像则下面的升级操作由 host 软件和系统共同完成，Ramboot 识别流程如图所述。

 版权所有 @ 上海富瀚微电子股份有限公司



图片 22: 升级过程

4.4 格式切换控制流程

格式切换的控制属于 VS 接口控制，请求交互为 VS 接口的 probe 和 commit 过程，probe 用于探测 VS 接口支持的适配格式带宽等属性参数，然后 commit 用于确认请求哪种参数的适配码流。具体的 probe 的探查过程流程和逻辑请参考 uvc 协议第 4.3.1.1 节 ``Video Probe and Commit Controls"。RTos uvc 的码流打开时 VS 接口的 probe 和 commit 过程抓包如图：

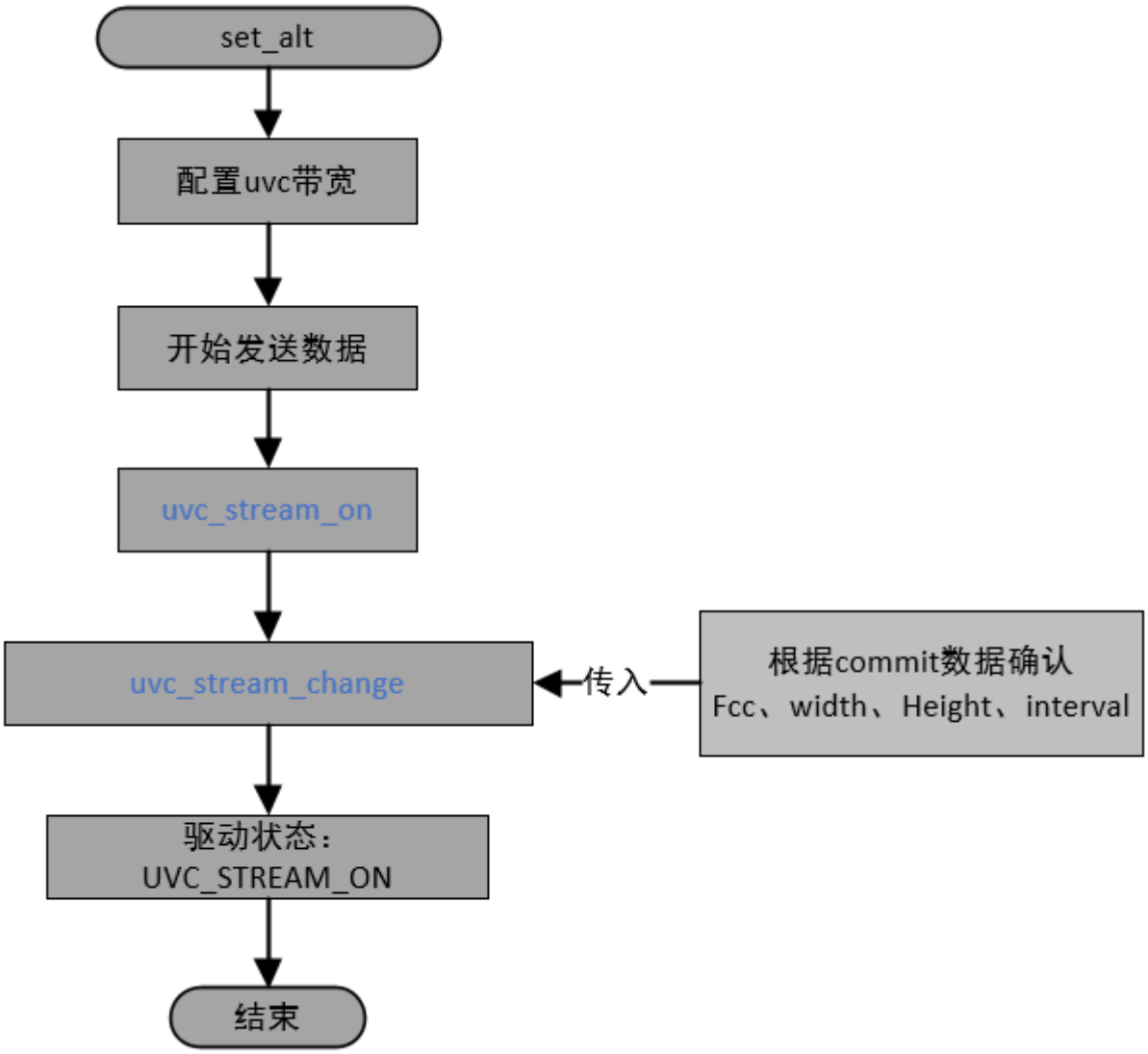
119 ms				[958 SOF]	[Frames: 2014.x - 85.6]
26 B	47	00	>	Get Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 01 00...)
83 ns				[1 SOF]	[Frame: 85.7]
26 B	47	00	>	Set Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
66 ns				[1 SOF]	[Frame: 86.0]
26 B	47	00	>	Get Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
26 B	47	00	>	Get Max	StreamInterfaceNum=1 CS=Probe(1) (01 00 05 06 15...)
66 ns				[1 SOF]	[Frame: 86.1]
26 B	47	00	>	Get Min	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 01 15...)
4.62 ms				[38 SOF]	[Frames: 86.2 - 90.7]
26 B	47	00	>	Get Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
26 B	47	00	>	Set Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
83 ns				[1 SOF]	[Frame: 91.0]
26 B	47	00	>	Get Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
66 ns				[1 SOF]	[Frame: 91.1]
26 B	47	00	>	Get Max	StreamInterfaceNum=1 CS=Probe(1) (01 00 05 06 15...)
26 B	47	00	>	Get Min	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 01 15...)
62.2 ms				[499 SOF]	[Frames: 91.2 - 153.4]
26 B	47	00	>	Set Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
66 ns				[1 SOF]	[Frame: 153.5]
26 B	47	00	>	Get Cur	StreamInterfaceNum=1 CS=Probe(1) (01 00 01 06 15...)
83 ns				[1 SOF]	[Frame: 153.6]
26 B	47	00	>	Set Cur	StreamInterfaceNum=1 CS=Commit(2) (01 00 01 06 15...)
375 us				[4 SOF]	[Frames: 153.7 - 154.2]
0 B	47	00	>	Set Interface	Interface=1 Alt. Setting=1

图片 23: probe 和 commit 过程

Probe 和 commit 对 VS 接口而言是不同的控件，下面仅以 SET_CUR commit 和 Set interface 1 alt=1 这个过程为例解释 uvc 是如何切换视频格式的。

commit 的信息是一个长度达到 42Byte(UVC1.5, 对于 UVC1.0 长度为 26Byte) 的码流配置数据，内容较多，我们在 uvc 中仅用到了视频格式、幅面、帧率和带宽的信息。因此收到 commit 请求后即根据 commit 配置信息确认主机请求的视频格式和带宽。其中对于格式和幅面的请求带宽由 uvc 设备在 GET_CUR 请求时发送给主机，即 probe 探测的时候告诉主机该幅面需要占用的代码，USB 带宽的选择由 uvc 幅面结构体指定。这里通俗点解释比如：主机将要请求 mjpeg-1080p-30 帧的视频数据，则先发送 GET_CUR probe 探查当前 uvc 配置的是哪种视频格式，然后发送 SET_CUR probe 将视频探查数据配置为 mjpeg-1080p-30 帧，此时 uvc 设备将按照该请求配置视频格式，并请求符合该视频格式的带宽，之后主机再发送 GET_CUR probe 探查之前 SET_CUR 配置下午的数据是否生效，如果生效其他格式是否符合主机要求，带宽要求能否满足能，主机检查 GET_CUR 数据没问题，则发送 SET_CUR commit 最终确认该格式的视频请求，使视频参数配置生效。

下图是主机最终请求切换 uvc vs 接口带宽，打开 uvc 视频传输的请求操作。Set interface1 Alt1 即为将 VS 接口从 0 带宽 (alt0) 切换到 Alt1 指定的带宽，这里 Alt1 的请求由主机根据 GET_CUR probe 的 dwMaxPayloadTransferSize 参数进行确定，可以是 Alt1~Alt8。



图片 24: 开启码流传输

配置带宽：将对应 alt 带宽的端点描述符指针赋值给 uvc->ep，该操作的目的是让 ep 的端点描述符跟随 alt 带宽而变化，尤其是端点大小，以保证后续视频传输时 usb 包大小的正确性。

开始发送数据：开始发送 uvc 报头信息，开启 uvc 码流端点的数据传输，uvc 数据传输，此时由于无码流数据，数据只有 uvc 报头，若判断码流 buf 链表中有视频数据，有则发送已含义报头的视频数据，无则只发送报头，之后所有数据传输 usb 完成中断中循环，直到关闭码流，vs 端点切换到 alt0。

uvc_stream_on：stream on 回调，可以自定义实现一些功能，如 LED 点亮等。

uvc_stream_change:将 SET_CUR commit 配置的视频格式参数传入到 format_change 回调函数中，对 isp、dsp 进行配置，改变捕捉的视频格式、幅面、帧率等信息。

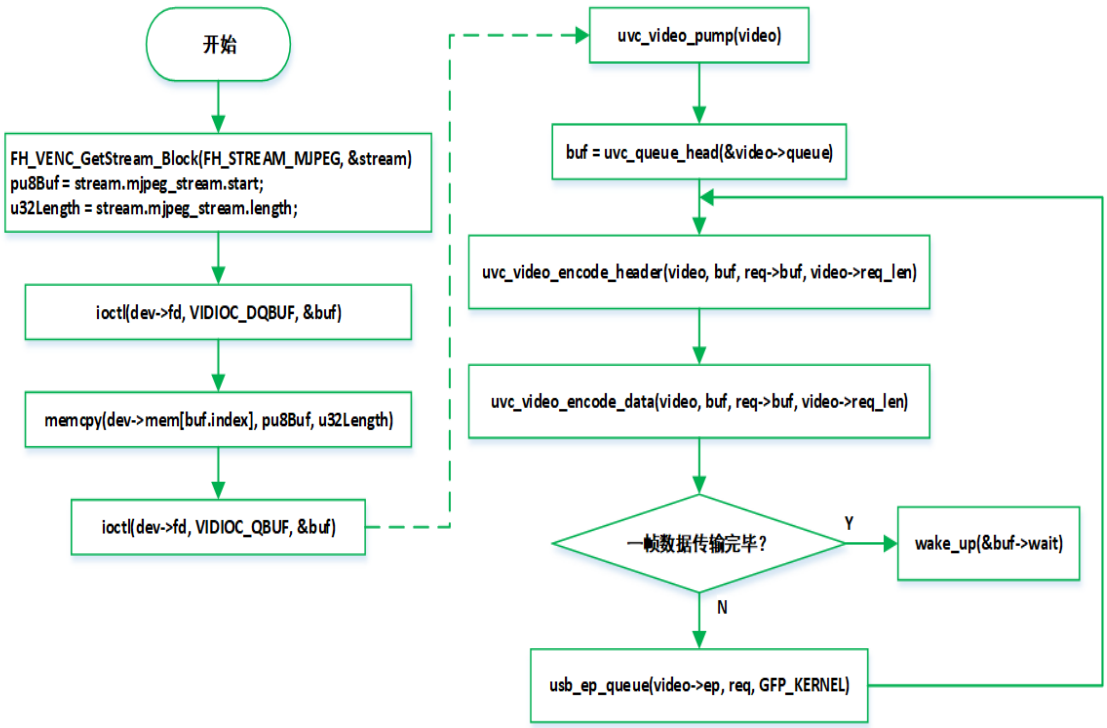
修改 uvc 状态：配置视频码流功能的状态，仅有码流开和码流关两种状态，打开码

流将状态配置为 UVC_STREAM_ON，此操作意味着 uvc 视频开始捕捉和传输。

4.5 UVC 数据传输流程

4.5.1 码流获取与编码

上一节中讲了格式切换控制流程，其中 set interface1 alt1 的配置中切换码流并将 uvc 码流状态配置为 UVC_STREAM_ON，可以理解为视频格式切换控制就是打开码流传输功能的控制操作。则本节介绍的是 uvc 的视频数据是如何传输的，在应用层代码中，一旦码流获取线程判断 uvc 状态为 UVC_STREAM_ON 即开始获取码流并向 uvc 驱动层传递。对于 Mjpeg、H264 和 H265 格式视频其码流流程如下：



图片 25: 码流捕获与传输

video_stream_proc：为码流传输的主要线程，所有格式视频 (包括 NV12 和 YUY2) 都通过该线程将视频码流数据推送到 uvc 驱动层，关于 NV12 和 YUY2 格式视频的处理过程在后续章节进行解释。

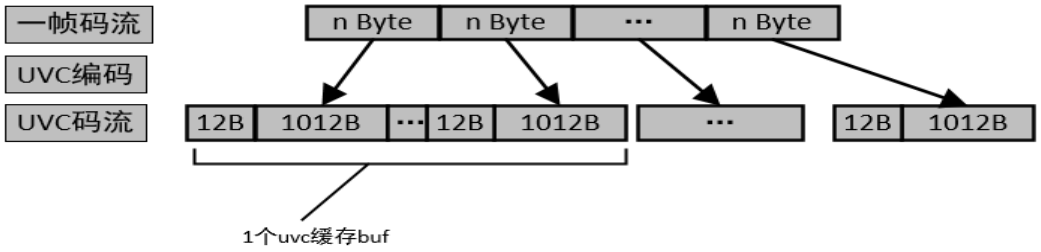
FH_VENC_GetStream_Block：以阻塞的方式获取视频数据，如果视频数据未就绪则等待，采用阻塞方式可以在获取码流的第一实际将数据 enqueue 到 uvc 驱动中，减少视频延时，

fh_uvc_stream_pts：将码流的 pts 信息配置到 uvc 驱动中，可以用于 uvc 报头 dwPresentationTime 字段填充。

 版权所有 @ 上海富瀚微电子股份有限公司

fh_uvc_stream_enqueue：将视频码流数据 buf 的地址和长度传入到 uvc 驱动中，如果 uvc 的缓存 buf 有空闲则取空闲的缓存 buf，然后对传入的视频按照 ``uvc 报头 + 视频数据" 的方式进行编码重组，重组后的数据存入空闲的缓存 buf 中，然后将缓存 buf 挂到缓存链表，uvc_video_complete 函数判断到视频缓存链表非空时则将编码后。的视频数据发送到 uvc 主机。

uvc_frame_encode：该函数处理是在 uvc 驱动中进行的，用户无需关系。将传入的视频数据按照 ``uvc 报头 + 视频数据" 的方式进行编码重组。以 ISOC ep 端点大小为 1024Byte 为例，则 uvc 视频数据端点每次可以传输 1024Byte 的有效数据，但是每个 uvc 报文都需要包含一个 2~12Byte 的报头数据，以报头长度为 12Byte 为例，则一个有效数据包为 ``12Byte header+ 1012Byte 码流数据"。如果一帧视频数据大小为 506KB，则需要传输 506*1024/1012=512 个数据包才可以将一帧数据传输完毕。一个 uvc 缓存 buf 的大小是可以自定义的 (宏定义 UVC_BUF_SIZE)，一般是一个 ep 大小的整数被，如 uvc 缓存 buf 为 100KB，则一个 buf 可以缓存 100 个 uvc 数据包，在发送时将 100 个数据包一次传输到 usb core 层驱动，可以增加传输效率。编码方式如下图：



图片 26: uvc 编码方式

4.5.2 YUV 码流处理

YUV 格式分为 NV12 和 YUY2 两种，目前支持的所有芯片仅 FH865x 是支持直接获取 YUY2 码流的，其他型号芯片都只能获取 NV12 格式的 YUV 数据然后转换到 YUY2 格式。对 YUY2 格式码流的处理，从获取 NV12 到转换为 YUY2 再到推流到 uvc 驱动层去做 uvc 编码，总共需要经历三个线程 yuv_get_proc、yuy2_proc 和 video_stream_proc。

yuv_get_proc：获取 NV12 码流，并将码流拷贝到 yuv_buf，然后将 buf 挂到 NV12 的缓存链表 list_used 上。

yuy2_proc：从 NV12 的缓存链表 list_used 上获取 nv12 码流，并将其转换为 YUY2 格式码流，存储到 yuy2_buf 上，然后将 buf 挂到 yuy2 的缓存链表 yuy2_used 上。

video_stream_proc：从 yuy2 的缓存链表 yuy2_used 上获取 YUY2 格式的码流数据并推流到 uvc 驱动层进行编码发送。

如果是 NV12 格式,则省略到 YUY2 的转换过程,总共经历两个线程,即 video_stream_proc 从 NV12 的缓存链表 list_used 上获取 nv12 码流并推流到 uvc 驱动层进行编码发送。

代码流程相对比较简单，主要是对各个阶段链表 buf 释放为空的判断，这里就不再画流程图展示，详细过程请参考 uvc_demo 代码。之所以将 YUV 码流分多个线程处理，

是为了 CPU 的利用最大化，采用``空间换时间" 的方法，加快 NV12 和 YUY2 格式码流的处理时间，减少传输延时和丢帧。

4.6 扩展功能

UVC 扩展功能如参数配置、固件导出、sensor hex 更新等功能均属于 uvc 控制流程的一部分，只是自定义了一些扩展功能函数，具体的功能实现可以根据上述对 uvc 协议及控制流程的介绍自行学习，本文不再进行介绍。

 版权所有 @ 上海富瀚微电子股份有限公司

5 Uvc demo 的使用和配置方法

Uvc_demo 的配置和使用方法，flash 固件的指针请参考《RT-Thread SDK 开发指南.pdf》第 11.2 节 ``UVC&UAC 使用指南"。

 版权所有 @ 上海富瀚微电子股份有限公司

6 UVC api

6.1 API 接口

6.1.1 fh_uvc_ops_register

原型

```
void fh_uvc_ops_register(struct uvc_stream_ops *ops)
```

简述

注册 uvc 操作和功能回调函数合集，所有应用层回调都在这里注册。

参数

类型	名称	描述	输入/输出
struct uvc_stream_ops	ops	回调函数结构体指针	in

6.1.2 fh_uvc_init

原型

```
void fh_uvc_init(int steram_id, struct uvc_format_info *fmt, int fmt_num)
```

简述

初始化 uvc 驱动

参数

类型	名称	描述	输入/输出
FH_INT32	steram_id	双 uvc 码流时用于码流选择,1~2	in
struct uvc_format_info	fmt	格式结构体地址	in
FH_INT32	fmt_num	uvc 格式数量	in

 版权所有 @ 上海富瀚微电子股份有限公司

6.1.3 fh_uvc_stream_enqueue

原型

```
void fh_uvc_stream_enqueue(int steram_id, uint8_t *img_data, uint32_t img_size);
```

简述

将视频数据发送到对于码流的 usb 端点

参数

类型	名称	描述	输入/输出
FH_INT32	steram_id	双 uvc 码流时用于码流选择,1~2	in
FH_UINT8 *	img_data	uvc 码流数据地址	in
FH_UINT32	img_size	uvc 码流数据长度	in

6.1.4 fh_uvc_stream_pts

原型

```
void fh_uvc_stream_pts(int stream_id, uint64_t pts)
```

简述

将视频的时间戳传到驱动中

参数

类型	名称	描述	输入/输出
FH_INT32	steram_id	双 uvc 码流时用于码流选择,1~2	in
FH_UINT64	pts	帧码流时间戳	in

6.1.5 fh_uvc_status_get

原型

```
int fh_uvc_status_get(int steram_id);
```

 版权所有 @ 上海富瀚微电子股份有限公司

简述

获取 uvc stream 驱动状态

参数

类型	名称	描述	输入/输出
FH_INT32	steram_id	双 uvc 码流时用于码流选择,1~2	in

返回值

2-stream on; 0-stream off

6.2 结构体定义

6.2.1 struct uvc_format_info

```
struct uvc_format_info
{
    unsigned int fcc;
    const struct uvc_frame_info *frames;
    unsigned int defaultindex;
};
```

类型	名称	描述	范围
FH_UINT32	fcc	uvc 视频格式	UVC_FORMAT 中的一种
struct uvc_frame_info	frames	幅面结构体数组的指针	
FH_UINT32	defaultindex	默认幅面 id	小于 frames 数组个数

6.2.2 struct uvc_frame_info

```
struct uvc_frame_info
{
    unsigned int width;
    unsigned int height;
    unsigned int intervals[8];
    unsigned int alt_set;
};
```

类型	名称	描述	范围
FH_UINT32	width	分辨率宽度	
FH_UINT32	height	分辨率高度	
FH_UINT32	intervals	该分辨率支持的帧率, 最多支持 8 种	
FH_UINT32	alt_set	指定带宽 id	1~8

6.2.3 struct uvc_stream_ops

```
struct uvc_stream_ops
{
    void (*format_change)(int stream_id, UVC_FORMAT fmt,
                                int w, int h, int fps);
    void (*stream_on_callback)(int stream_id);
    void (*stream_off_callback)(int stream_id);
    void (*uvc_extern_data)(unsigned char *data,
            unsigned int data_size, unsigned int cs);
    void (*uvc_extern_intr)(unsigned char req,
            unsigned char cs, struct uvc_request_data *resp);
    struct UvcInfo *(*uvc_vision)(void);
    int *(*uvc_uspara)(void);

    /* PROCESSING_UNIT_CONTROLS callback func */
    void (*SetGain)(unsigned int gain_level);       /* 增益 */
    void (*SetBrightness)(unsigned int value);      /* 亮度 */
    void (*SetContrast)(unsigned int value);        /* 对比度 */
    void (*SetSaturation)(int value);               /* 饱和度 */
    void (*SetSharpeness)(int value);               /* 清晰度 */
    void (*SetAWBGain)(unsigned int value);         /* 白平衡 */
    void (*SetAwbMode)(unsigned int value);         /* 白平衡 0: 手动, 1: 自动 */
    void (*SetGammaCfg)(unsigned int value);        /* 伽马 */
    void (*SetBacklight)(unsigned int value);       /* 逆光对比 */
    void (*SetHueAuto)(unsigned int value);         /* 色调 0: 手动, 1: 自动 */
    void (*SetHueGain)(unsigned int value);         /* 色调 */
    void (*SetPowerLine)(unsigned int value);       /* 电力线 1:50hz, 2:60hz*/

    /* CAMERA_TERMINAL_CONTROLS callback func */
    void (*SetAEMode)(unsigned int mode);           /* 1: 曝光手动, 光圈手动; */
                                                    /* 2: 曝光自动, 光圈自动; */
                                                    /* 4: 曝光手动, 光圈自动; */
                                                    /* 8: 曝光自动, 光圈手动 */
    void (*SetExposure)(unsigned int time_value);   /* 曝光时间 */
    void (*SetIris)(unsigned int value);            /* 光圈 */
    void (*SetAe_Priority)(unsigned int value);     /* 低亮度补偿:enable/disable */
    void (*SetFocusMode)(unsigned int value);       /* 焦点模式 1: 自动, 0: 手动 */
    void (*SetFocusAbs)(unsigned int value);        /* 焦点 */
    void (*SetZoomAbs)(unsigned int value);         /* 缩放 */
    void (*SetPanTiltAbs)(int pan, int tilt);       /* 全景 & 倾斜 */
    void (*SetRollAbs)(int value);                  /* 滚动 */
    int reserved[12];
};
```

类型	名称	描述	范围
void	format_change	格式配置操作函数，用于改变视频格式和幅面参数	
void	stream_on_callback	打开 uvc 视频回调	
void	stream_off_callback	关闭 uvc 视频回调	
void	uvc_extern_data	uvc 扩展单元的 Set 请求回调函数	
void	uvc_extern_intr	uvc 扩展单元的 Get 请求回调函数	
struct UvcInfo	uvc_vision	从 flash 读取 uvc 产品信息	
FH_INT32	uvc_uspara	从 flash 读取 uvc 视频参数信息	
void	SetGain	增益配置回调	
void	SetBrightness	亮度配置回调	
void	SetContrast	对比度配置回调	
void	SetSaturation	饱和度配置回调	
void	SetSharpeness	清晰度配置回调	
void	SetAWBGain	白平衡配置回调	
void	SetAwbMode	白平衡手自动模式切换回调	0/1
void	SetGammaCfg	伽马配置回调	
void	SetBacklight	逆光对比配置回调	
void	SetHueAuto	色调手自动模式切换回调	0/1
void	SetHueGain	色调增益配置回调	
void	SetPowerLine	电力线控制回调	50hz/60hz
void	SetAEMode	曝光模式切换回调	1/2/4/8
void	SetExposure	曝光时间配置回调	
void	SetIris	光圈配置回调	
void	SetAe_Priority	低亮度补充配置回调	
void	SetFocusMode	手自动对焦模式配置回调	0/1
void	SetFocusAbs	对焦配置回调	
void	SetZoomAbs	缩放配置回调	
void	SetPanTiltAbs	全景 & 倾斜配置回调	
void	SetRollAbs	滚动配置回调	

6.2.4 struct UvcInfo

```
struct UvcInfo
{
    unsigned int size;
    unsigned int crc;
    unsigned int uvcinfo_change;

    union _Info
    {
        struct InfoData
        {
            unsigned char data[1012];
        } infoData;

        struct InfoVal
        {
            unsigned int vid;
            unsigned int pid;
            unsigned int bcd;
```

```
            char szFacture[32];
            char szProduct[32];
            char szSerial[32];
            char szVideoIntf[32];
            char szMicIntf[32];
        } infoVal;
    } info;
} __packed;
```

类型	名称	描述
FH_UINT32	size	有效数据长度不包含 size 和 crc 字段
FH_UINT32	crc	有效数据 crc 校验值
FH_UINT32	uvcinfo_change	标志位, 1：flash 有 uvc 版本信息，0：flash 中尚未保存 uvc 信息
FH_UINT32	vid	设备 idVendor
FH_UINT32	pid	设备 idProduct
FH_UINT32	bcd	设备 bcd 版本
FH_UINT8	szFacture	产品厂家信息
FH_UINT8	szProduct	产品型号
FH_UINT8	szSerial	产品序列号
FH_UINT8	szVideoIntf	uvc 设备名称
FH_UINT8	szMicIntf	uac 设备名称

6.2.5 struct uvc_request_data

```
struct uvc_request_data
{
    int length;
    unsigned char data[60];
};
```

类型	名称	描述
FH_INT32	length	请求输入或输出的数据长度
FH_UINT8	data	uvc 控制请求输入或输出的数据

6.2.6 UVC_FORMAT

```
typedef enum
{
```

```
    V4L2_PIX_FMT_NV12   = v4l2_fourcc('N', 'V', '1', '2'),
    V4L2_PIX_FMT_YUY2   = v4l2_fourcc('Y', 'U', 'Y', '2'),
    V4L2_PIX_FMT_MJPEG  = v4l2_fourcc('M', 'J', 'P', 'G'),
    V4L2_PIX_FMT_H264   = v4l2_fourcc('H', '2', '6', '4'),
    V4L2_PIX_FMT_H265   = v4l2_fourcc('H', '2', '6', '5'),
    V4L2_PIX_FMT_reserved1   = v4l2_fourcc('0', '0', '0', '0'),
    V4L2_PIX_FMT_reserved2   = v4l2_fourcc('0', '0', '0', '0')
} UVC_FORMAT;
```

名称	描述
V4L2_PIX_FMT_NV12	NV12 格式标识
V4L2_PIX_FMT_YUY2	YUY2 格式标识
V4L2_PIX_FMT_MJPEG	Mjpeg 格式标识
V4L2_PIX_FMT_H264	H264 格式标识
V4L2_PIX_FMT_H265	H265 格式标识
V4L2_PIX_FMT_reserved1	保留
V4L2_PIX_FMT_reserved2	保留

 版权所有 @ 上海富瀚微电子股份有限公司